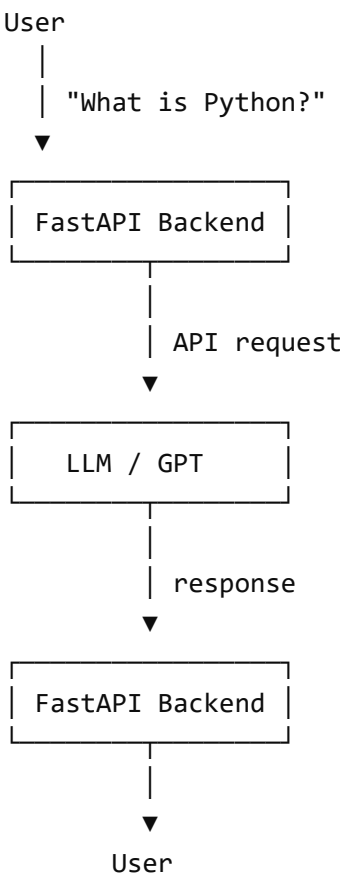
> How to build an AI chat bot? can you explain me with example step by step, so I will build it with your guidance

Absolutely. Since you're a Python backend developer, I'd recommend that we **build the chatbot ourselves step by step**, rather than starting with LangChain or other frameworks immediately. That way you'll understand what is actually happening underneath.

We can eventually evolve the same project into a **production-style AI agent with RAG, tools, memory, streaming, authentication, and deployment**.

What we're going to build

Let's start with a simple chatbot:



Then we'll progressively make it more powerful:

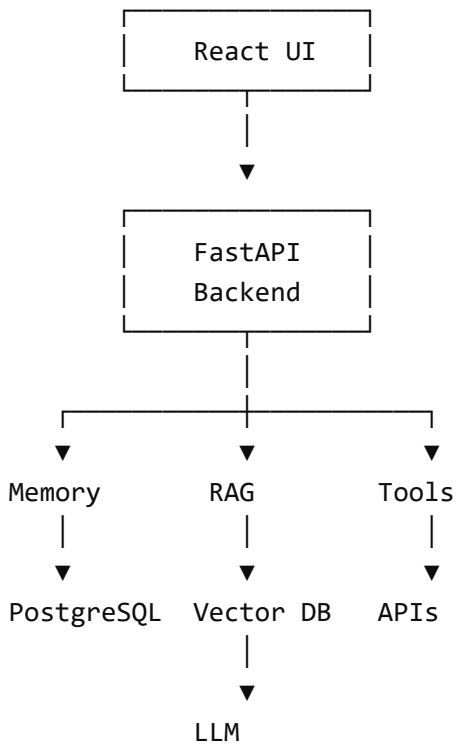
```
Level 1
Basic chatbot
    ↓
Level 2
Conversation history / memory
```

Upgrade

```
Level 3
Streaming responses
    ↓
Level 4
RAG — chat with your documents
    ↓
Level 5
Vector database
    ↓
Level 6
Tools / function calling
    ↓
Level 7
AI Agent
    ↓
Level 8
Production architecture
```

The final project

Eventually, we'll have something like:



Step 1 — Understand what an AI chatbot actually is

The most important thing to understand first:

A chatbot is not the LLM itself.

An LLM is basically the component that generates the response.

For example:

```
User:
"What is Python?"
```

Your application sends something like:

```
messages = [
    {
        "role": "user",
        "content": "What is Python?"
    }
]
```

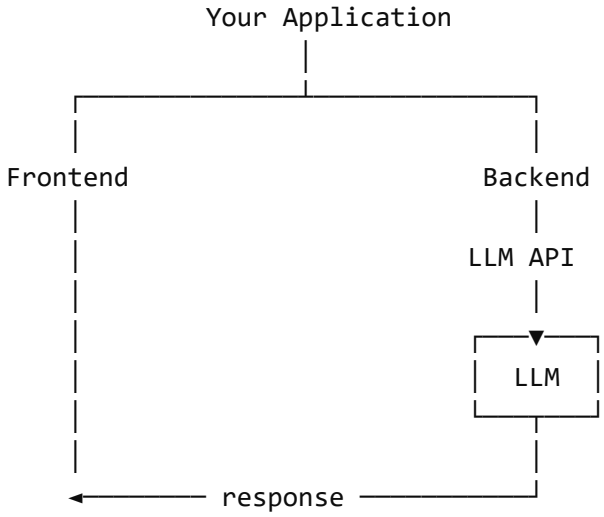
to an LLM.

The LLM generates:

```
Python is a high-level programming language...
```

Your backend then sends that response back to the frontend.

So conceptually:



Step 2 — What technologies should we use?

Because you're already comfortable with Python and FastAPI, let's use:

Component	Technology
Language	Python
Backend	FastAPI
LLM	OpenAI API initially
Database	PostgreSQL
Conversation memory	PostgreSQL
RAG later	Qdrant
Embeddings	OpenAI embeddings
Frontend	React
Containerization	Docker
Deployment	We'll decide later

I'm deliberately **not starting with LangChain**.

First we'll understand:

```
Python
   ↓
LLM API
   ↓
Chat history
   ↓
RAG
   ↓
Tools
   ↓
Agent
```

Then you'll understand what frameworks like LangChain/LangGraph are actually doing for you.

Step 3 — Create the project

Create a directory:

Bash

```
mkdir ai-chatbot
cd ai-chatbot
```

Create a virtual environment:

Bash

```
python -m venv venv
```

Activate it.

Windows

Bash

```
venv\Scripts\activate
```

Install the initial packages:

Bash

```
pip install fastapi uvicorn openai python-dotenv
```

Our first project structure:

```
ai-chatbot/
│
├── venv/
│
├── app/
│   ├── __init__.py
│   └── main.py
│
├── .env
├── .gitignore
└── requirements.txt
```

Generate requirements:

Bash

```
pip freeze > requirements.txt
```

Step 4 — Get an LLM API key

We'll use an API-based LLM initially.

Create:

```
.env
```

and put your API key there:

```
OPENAI_API_KEY=your_api_key_here
```

Don't put the API key directly into Python code.

Bad:

```python
client = OpenAI(
    api_key="sk-xxxxxxxx"
)
```

Good:

```
.env
    ↓
environment variable
    ↓
Python
```

And make sure `.env` is in `.gitignore`:

```
.env
venv/
__pycache__/
```

Step 5 — Make your first LLM call

Create:

```
app/main.py
```

Start with this:

Python

Run

```python
import os

from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY")
)

response = client.responses.create(
    model="gpt-5",
    input="What is Python?"
)

print(response.output_text)
```

Run:

Bash

```bash
python app/main.py
```

You should get an answer generated by the model.

What happened here?

This:

Python

Run

```python
client = OpenAI(...)
```

creates an API client.

Then:

Python

Run

```python
client.responses.create(...)
```

sends a request to the model.

And:

Python

Run

```
response.output_text
```

gets the generated text.

That's the **core of an LLM application**.

Everything else we're going to build is around this.

Step 6 — Turn it into a FastAPI API

Now we'll make our own backend.

Replace `main.py` with:

Python

Run

```
import os

from dotenv import load_dotenv
from fastapi import FastAPI
from openai import OpenAI
from pydantic import BaseModel

load_dotenv()

app = FastAPI()

client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY")
)


class ChatRequest(BaseModel):
    message: str


@app.post("/chat")
def chat(request: ChatRequest):

    response = client.responses.create(
        model="gpt-5",
        input=request.message
    )

    return {
        "response": response.output_text
```

```
    }
```

Start FastAPI:

```bash
uvicorn app.main:app --reload
```

Now open:

```
http://127.0.0.1:8000/docs
```

You'll see Swagger UI.

Call:

```
POST /chat
```

with:

```json
{
    "message": "Explain Python decorators"
}
```

Your API will return:

```json
{
    "response": "A decorator in Python is..."
}
```

🎉 **You have built your first AI chatbot backend.**

Step 7 — But this isn't really a chatbot yet

Here's an important concept.

Suppose the user says:

```
User:
My name is Vijay.
```

LLM responds:

```
Nice to meet you, Vijay!
```

Then user asks:

```
What is my name?
```

Our current application sends only:

```
"What is my name?"
```

The model may not know.

Why?

Because **LLMs don't automatically remember previous API requests.**

Our application needs to provide the conversation history.

For example:

```
messages = [
    {
        "role": "user",
        "content": "My name is Vijay."
    },
    {
        "role": "assistant",
        "content": "Nice to meet you, Vijay!"
    },
    {
        "role": "user",
        "content": "What is my name?"
    }
]
```

Now the model has the context it needs.

This leads us to our next major concept.

Step 8 — Add conversation memory

Initially, we can keep the history in Python:

```
Python                                                    Run
```

```
conversation = []
```

When the user sends:

```
"My name is Vijay"
```

we add:

Python Run

```python
conversation.append({
    "role": "user",
    "content": "My name is Vijay"
})
```

Then after getting the response:

Python Run

```python
conversation.append({
    "role": "assistant",
    "content": response.output_text
})
```

Next request includes the entire conversation.

Conceptually:

```
Request 1

User
 |
 ▼
"My name is Vijay"
 |
 ▼
LLM
 |
 ▼
"Nice to meet you Vijay"
 |
 ▼
Memory
```

Then:

```
Request 2
```

```
User
 │
 ▼
"What is my name?"
 │
 ▼
Memory
 │
 ├── My name is Vijay
 ├── Nice to meet you Vijay
 │
 ▼
LLM
 │
 ▼
"Your name is Vijay"
```

But storing it in a Python list isn't suitable for a real application.

If your server restarts:

```
conversation = []
```

💥 Memory disappears.

So eventually we'll use:

```
PostgreSQL
```

for conversation persistence.

Step 9 — The first major architecture

After implementing memory:

```
                User
                 │
                 ▼
            React UI
                 │
                 ▼
             FastAPI
                 │
        ┌────────┴────────┐
        │                 │
        ▼                 ▼
   PostgreSQL          LLM
   conversation         │
```

```
        history                 │
           │                    │
           └────────────────────┘
                     │
                     ▼
                 Response
```

Now we have a real chatbot architecture.

Step 10 — Then we'll teach the chatbot about your documents

This is where things become much more interesting.

Suppose you upload:

```
company_policy.pdf
```

and ask:

> What is the leave policy?

The LLM doesn't automatically know your company's PDF.

We need **RAG — Retrieval-Augmented Generation**.

The flow becomes:

```
PDF
 │
 ▼
Extract text
 │
 ▼
Split into chunks
 │
 ▼
Create embeddings
 │
 ▼
Vector Database
```

Then when user asks:

```
"What is the leave policy?"
```

we do:

```
Question
   |
   ▼
Embedding
   |
   ▼
Vector DB
   |
   ▼
Relevant chunks
   |
   ▼
LLM
   |
   ▼
Answer
```

This is one of the most important architectures you'll need to understand as an AI engineer.

Step 11 — Then we'll add tools

Imagine the user asks:

> What is the weather in Hyderabad?

The LLM itself shouldn't necessarily invent the current weather.

Instead:

```
User
 |
 ▼
LLM
 |
 | decides:
 | "I need weather information"
 ▼
Weather Tool
 |
 ▼
Weather API
 |
 ▼
Result
 |
```

```
 ▼
LLM
 │
 ▼
Final answer
```

Or:

> What is the status of order 12345?

The agent could call:

```
get_order_status(12345)
```

Or:

> Send an email to John.

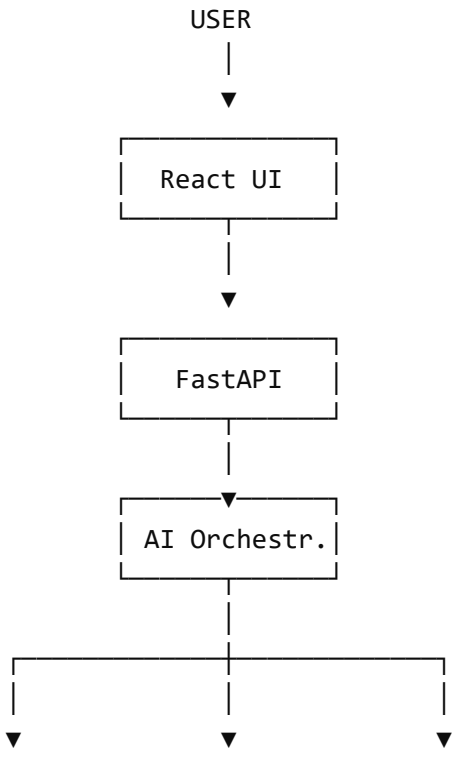
It could call:

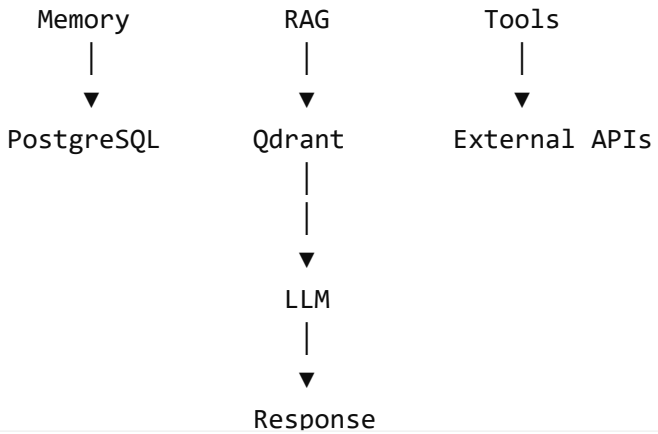
```
send_email(...)
```

Now you're entering **AI agent territory**.

Step 12 — Eventually we'll build this

Our final architecture will look approximately like:





And then we'll add:

```
Streaming
Authentication
Rate limiting
Caching
Observability
Error handling
Prompt management
Token management
Evaluation
Docker
Kubernetes
```

So you'll learn not just **"how to call ChatGPT from Python"**, but how to build an **AI-powered backend application**.

Our learning/building sequence

I suggest we follow this exact order:

Phase 1 — LLM fundamentals

1. What is an LLM?
2. Tokens
3. Context window
4. System/user/assistant messages
5. Temperature and model parameters
6. API calls
7. Structured output

Phase 2 — Build chatbot

8. FastAPI + LLM
9. Conversation history

10. Persistent memory
11. PostgreSQL
12. Streaming responses
13. React chat UI

Phase 3 — RAG

14. Embeddings
15. Vector databases
16. Chunking
17. Retrieval
18. RAG pipeline
19. PDF/document chatbot
20. Qdrant

Phase 4 — Tool calling

21. Function calling
22. Tools
23. External APIs
24. Database tools
25. Tool selection

Phase 5 — AI Agents

26. What is an agent?
27. Agent loop
28. Planning
29. Tool execution
30. State management
31. LangGraph
32. Multi-agent systems

Phase 6 — Production

33. Authentication
34. Rate limiting
35. Caching
36. Token/cost management
37. Logging
38. Observability
39. Evaluation
40. Docker
41. Kubernetes

42. Production architecture

Most important: let's build it together

Don't try to build all of this at once.

We'll actually write the code together one step at a time.

For your first exercise, do only **Steps 3–6**:

```
1. Create ai-chatbot/
2. Create virtual environment
3. Install packages
4. Configure OPENAI_API_KEY
5. Make the first LLM call
6. Create POST /chat
7. Test it through Swagger
```

Once you have `POST /chat` **working**, tell me **"Step 1 completed"** and we'll move to the next stage: **understanding exactly how conversation history works and turning this basic API into a real multi-turn chatbot**.

That approach will also make it much easier later when we move from chatbot → **RAG** → **tools** → **AI agent**.

ChatGPT can make mistakes. Check important info.